Section 2.3: Encryption, TLS & Secure Transport

> ***Enhanced Edition*** *— Additional examples, comparisons, and interview preparation*

Executive Overview

Transport security protects data in motion. Understanding TLS handshakes, mutual authentication, certificate lifecycle management, and secure protocol selection is essential for any system design interview involving sensitive data, public APIs, or microservice communication.

The core mental model: Security in transit has two goals — **confidentiality** (nobody can read the data) and **authenticity** (you're talking to who you think you are). TLS achieves both. mTLS extends authenticity to both sides of the connection. Understanding which you need, and at what cost, is what interviewers probe.

SSL/TLS Deep Dive

What TLS Actually Does — Three Guarantees

```
1. Confidentiality:   Data encrypted with symmetric key (AES-GCM)
                      Eavesdropper sees ciphertext only

2. Integrity:         AEAD (Authenticated Encryption with Associated Data)
                      Any modification to ciphertext is detected

3. Authentication:    Server presents X.509 certificate
                      Client verifies: signed by trusted CA? CN/SAN matches?
                      (mTLS adds: client also presents certificate)
```

Without all three, you're vulnerable: - No confidentiality → wiretapping - No integrity → bit-flip attacks on ciphertext - No authentication → man-in-the-middle (MITM) substitutes their key

TLS 1.2 vs. TLS 1.3 — Handshake by Handshake

TLS 1.2 (2-RTT, pre-2018):

```
Client                                          Server
  |── ClientHello ──────────────────────────→ |
  |   (supported cipher suites, random nonce)   |
  |                                             |
  | ←─ ServerHello ──────────────────────────── |
  |    (chosen cipher, server random nonce)     |
  | ←─ Certificate ──────────────────────────── |
  |    (server's X.509 cert)                    |
  | ←─ ServerKeyExchange ────────────────────── |  ← only for DHE/ECDHE
  |    (DH public key, signed by cert)          |
  | ←─ ServerHelloDone ──────────────────────── |
  |                                             |
  |── ClientKeyExchange ──────────────────── → |
  |   (client DH public key or encrypted PSK)   |
  |── ChangeCipherSpec + Finished ──────────→ |
  |                                             |
  | ←─ ChangeCipherSpec + Finished ──────────── |
  |                                             |
  |══ Application Data ═══════════════════════ |

  Total: 2 RTT before first application byte
  With cross-continent RTT of 80ms: 160ms of TLS overhead alone
```

TLS 1.3 (1-RTT, 2018 RFC 8446):

```
Client                                          Server
  |── ClientHello ──────────────────────────→ |
  |   (key_share: client DH public key          |
  |    supported_groups, cipher suites)         |
  |                                             |
  | ←─ ServerHello ──────────────────────────── |
  |    (key_share: server DH public key)        |
  | ←─ Certificate + CertificateVerify ──────── |
  | ←─ Finished ─────────────────────────────── |
  |                                             |
  |── Finished ──────────────────────────────→ |
  |══ Application Data ═══════════════════════ |

  Total: 1 RTT before first application byte
  Savings: 1 full RTT (80ms cross-continent) vs TLS 1.2

Key change: Client sends DH key share in ClientHello (speculates on algorithm)
Server can derive shared secret immediately → Finished in same flight as cert
```

TLS 1.3 0-RTT Resumption (session tickets):

```
Previous session established session ticket (encrypted resumption state)

Client                                          Server
  |── ClientHello + early_data ──────────────→ |
  |   (session ticket + application data)       |
```

```
  |                                                  |
  | ←— ServerHello + Finished ——————————————— |
  |== Application Data continues ================ |

  Total: 0 RTT — application data sent in first packet!

SECURITY CAVEAT: 0-RTT data has no forward secrecy and is
replay-attackable. Never use 0-RTT for non-idempotent operations.
```

The Full Latency Comparison

```
Protocol Stack                Handshakes     Cross-Continent Cost
————————————————————————————————————————————————————————————

TCP only (no TLS)             1 RTT          80ms
TCP + TLS 1.2 (new)           3 RTT          240ms
TCP + TLS 1.2 (resumed)       3 RTT          240ms  (no session resumption)
TCP + TLS 1.3 (new)           2 RTT          160ms
TCP + TLS 1.3 (resumed 1-RTT) 1 RTT          80ms
TCP + TLS 1.3 (0-RTT)         0 RTT          ~5ms   (data in flight immediately)
QUIC + TLS 1.3 (new)          1 RTT          80ms   (TCP+TLS combined)
QUIC + TLS 1.3 (0-RTT)        0 RTT          ~5ms
```

Cipher Suite Selection

Why AEAD ciphers replaced CBC-mode ciphers:

Old (CBC + HMAC — “encrypt-then-MAC” or broken “MAC-then-encrypt”):

```
Vulnerability: Padding oracle attacks (POODLE, BEAST, Lucky 13)
  Attacker manipulates ciphertext, observes error timing differences
  → Decrypts ciphertext one byte at a time

Historical victims: SSL 3.0 (POODLE), TLS 1.0 (BEAST), TLS 1.2 with CBC (Lucky 13)
```

Modern (AEAD — encryption and authentication in one pass):

```
AES-256-GCM:
  - AES in Galois/Counter Mode
  - 128-bit authentication tag appended to ciphertext
  - Any modification (even 1 bit) → authentication fails → connection closed
  - Hardware acceleration: AES-NI on x86 → ~1 GB/s per core

ChaCha20-Poly1305:
  - Stream cipher (no padding) + Poly1305 MAC
  - Software-optimized: faster than AES on ARM without AES hardware acceleration
```

```
  - Used by: Android (many chips lack AES-NI), IoT devices, lower-end servers
  - Google switched mobile Chrome to ChaCha20 → 30% TLS CPU reduction on Android
```

TLS 1.3 cipher suites (only AEAD allowed — weak ciphers removed):

```
TLS_AES_256_GCM_SHA384:        # Highest security, slower
TLS_AES_128_GCM_SHA256:        # Default balance of security and speed
TLS_CHACHA20_POLY1305_SHA256:  # Mobile/ARM optimized
```

Removed in TLS 1.3 (all considered weak or broken):

```
RC4, DES, 3DES, MD5, SHA-1
RSA key exchange (no forward secrecy)
CBC mode ciphers (padding oracle risk)
Export-grade ciphers (FREAK attack target)
```

Forward Secrecy — Why It Matters

Without forward secrecy (RSA key exchange, deprecated):

```
Attack scenario:
  1. Attacker records encrypted traffic today (passive wiretap)
  2. Server's private key leaked 5 years later
  3. Attacker decrypts ALL recorded traffic retroactively

Real incident: NSA PRISM program reportedly recorded encrypted traffic
               at scale, presumably to decrypt when keys were obtained
```

With forward secrecy (ECDHE — Ephemeral Elliptic Curve Diffie-Hellman):

```
Key exchange process:
  1. Client generates ephemeral key pair (fresh every handshake)
  2. Server generates ephemeral key pair (fresh every handshake)
  3. Both derive shared session key via DH math
  4. Ephemeral keys are DISCARDED after session

Even if server's long-term private key is compromised:
  Attacker cannot derive past session keys (they were never stored)
  Only that specific handshake moment can be compromised

"Ephemeral" is the critical word — key only exists for one session
```

Diffie-Hellman Key Exchange — the math intuition:

```
Public information: prime p, generator g (agreed in ClientHello)
Client private key:  a (secret, never transmitted)
```

```
Server private key:  b (secret, never transmitted)

Client sends: A = g^a mod p  (public)
Server sends: B = g^b mod p  (public)

Client computes: B^a mod p = g^(ba) mod p
Server computes: A^b mod p = g^(ab) mod p
Both arrive at same shared secret: g^(ab) mod p

Eavesdropper sees A and B but cannot compute g^(ab) without knowing a or b
(Discrete logarithm problem: computationally infeasible to reverse g^a → a)
```

Certificate Management

X.509 Certificate Anatomy

```
Certificate {
  Version:          3 (X.509v3)
  Serial Number:    0x1A2B3C4D5E6F (unique per CA)
  Signature Alg:    sha256WithRSAEncryption (or ECDSA)
  Issuer:           CN=DigiCert Global CA G2, O=DigiCert Inc
  Validity:
    Not Before:     2024-01-15 00:00:00 UTC
    Not After:      2025-01-15 23:59:59 UTC    ← 1-year max (post-2020)
  Subject:          CN=api.example.com, O=Example Corp
  Public Key:       RSA 2048-bit (or ECDSA P-256)
  Extensions:
    SubjectAltName: DNS:api.example.com, DNS:*.example.com  ← actual hostname
matching
    KeyUsage:       digitalSignature, keyEncipherment
    ExtKeyUsage:    serverAuth, clientAuth
    BasicConstraints: CA:FALSE  ← cannot issue other certs
    OCSP URL:       http://ocsp.digicert.com
    CRL URL:        http://crl.digicert.com/DigiCertGlobalCAG2.crl
}
Signed by CA's private key → tamper-evident
```

Why SANs matter more than CNs:

```
Old behavior (deprecated): Check Common Name (CN) field for hostname
Current behavior: Check Subject Alternative Name (SAN) only

If your cert has CN=api.example.com but no SAN entry:
  Modern browsers/clients reject it — hostname mismatch

Wildcard certs: *.example.com matches:
  api.example.com ✓
  www.example.com ✓
```

```
  sub.api.example.com ✗  (only one level deep)
  example.com ✗          (must have explicit SAN entry)
```

Certificate Chain of Trust

```
Root CA (self-signed, stored in OS/browser trust store)
    └── Intermediate CA (signed by Root)
            └── Leaf Certificate (your server cert, signed by Intermediate)

Verification process:
  1. Client receives leaf cert + intermediate cert(s) from server
  2. Verifies leaf cert signature using intermediate's public key
  3. Verifies intermediate cert signature using root's public key
  4. Root is in OS trust store → chain trusted

Why intermediates?
  Root CA private key is kept offline (HSM in a vault, never touches internet)
  Intermediate CA key is online but if compromised: revoke just the intermediate
  Root compromise = catastrophic (re-trust 5+ years)
```

Certificate Revocation — the Reliability Problem

CRL (Certificate Revocation List):

```
CA publishes list of revoked serial numbers as a file
Client downloads CRL, checks if cert's serial is listed

Problem:
  CRL files can be megabytes (millions of revocations)
  Client downloads on every new session → high latency
  CRL has a publish interval (daily) → stale for up to 24h
  If CRL server is down: fail open (most clients) or fail closed
```

OCSP (Online Certificate Status Protocol):

```
Client → OCSP Responder: "Is serial 0x1A2B3C4D revoked?"
OCSP Responder → Client: "Good / Revoked / Unknown"

Problem:
  Extra RTT per TLS handshake (OCSP server lookup)
  OCSP server is a privacy leak (CA learns which sites you visit)
  Availability dependency: if CA's OCSP is down, what do you do?
  Most browsers "fail open" (allow) if OCSP unreachable → security theater
```

OCSP Stapling (current best practice):

```
Server fetches its own OCSP response from CA (periodically, e.g., every hour)
Server "staples" the signed OCSP response to its TLS handshake

Client receives: Certificate + Stapled OCSP response (already signed by CA)
Client verifies: OCSP response signature + timestamp (must be recent)

Benefits:
  No extra RTT for OCSP check (bundled with handshake)
  No privacy leak (client never contacts CA directly)
  CA OCSP server availability doesn't affect your service

Must-Staple extension:
  Cert includes "must-staple" flag → client rejects if no staple present
  Prevents downgrade (attacker strips staple)
```

Let's Encrypt and 90-day cert lifecycles:

```
Traditional CAs: 1-2 year certificates, manual renewal
Let's Encrypt: 90-day certificates, automated via ACME protocol

ACME protocol (RFC 8555):
  certbot renew:
  1. Client proves domain control (HTTP-01: serve token at /.well-known/acme-
challenge/)
  2. CA validates → issues 90-day cert
  3. Scheduled via cron/systemd timer (renew at 60 days)

90-day rationale:
  Shorter validity = faster revocation effectiveness (cert expires before CRL
staleness matters)
  Forces automation (can't forget to renew for 2 years then panic)
  Compromise window: 90 days max vs 2 years
```

Mutual TLS (mTLS)

What mTLS Adds Over Regular TLS

Standard TLS (one-way authentication):

```
Server: "Here is my certificate — I am api.example.com"
Client: Verifies server cert → encrypted channel
Result: Client knows it's talking to a real server
        Server has NO IDEA who the client is (just an IP address)
```

mTLS (bidirectional authentication):

```
Server: "Here is my certificate — I am api.example.com"
Client: "Here is MY certificate — I am order-service in cluster prod-us-east"
Both:   Verify each other's certs against a shared CA
Result: Cryptographic proof of identity in BOTH directions
        No password, no API key — identity is baked into the TLS layer
```

Why mTLS for microservices:

```
Problem with API keys / bearer tokens between services:
  - Secret stored in environment variable → exfiltration risk
  - Rotation requires coordinated deploy across services
  - Stolen key grants access from anywhere
  - No cryptographic binding to the process using it

mTLS advantages:
  - Private key never leaves the process (generated on-node)
  - Certificate tied to workload identity (SPIFFE ID)
  - Rotation automated (SPIRE, cert-manager)
  - Stolen cert is useless without private key (which was never exported)
  - Mutual: server also verified, prevents rogue service impersonation
```

mTLS Handshake — the Additional Steps

```
Client                                          Server
  |── ClientHello ──────────────────────────→ |
  |                                           |
  | ←─ ServerHello + Certificate ──────────── |
  | ←─ CertificateRequest  ─────────────────── |  ← NEW: server demands client
cert
  |                                           |
  |── Certificate (client cert) ────────────→ |  ← NEW: client sends its cert
  |── CertificateVerify (signature) ────────→ |  ← NEW: proves it holds private
key
  |── Finished ─────────────────────────────→ |
  |                                           |
  | ←─ Finished ──────────────────────────── |
  |══ Application Data ═══════════════════════ |

Additional overhead vs. TLS:
  Extra cert transmission: ~2-4 KB
  Extra signature verification: ~0.1ms
  Negligible for service-to-service (internal latency already ~1ms)
```

SPIFFE/SPIRE Framework — Workload Identity at Scale

The problem SPIFFE solves:

```
Traditional approach:
  Service A needs to call Service B
  → Store API key in Kubernetes secret
  → Secret accessible to any pod in namespace
  → Rotated by humans on some schedule
  → Leaked key = compromised until discovered

SPIFFE approach:
  Service A automatically receives an SVID (SPIFFE Verifiable Identity Document)
  → SVID is a short-lived X.509 cert (1-24h TTL)
  → Cert tied to Kubernetes service account (attested by node agent)
  → Private key generated on-node, never leaves
  → Rotated automatically before expiry
  → No human involvement after initial configuration
```

SPIFFE ID format:

```
spiffe://trust-domain/path

Examples:
  spiffe://prod.example.com/ns/payments/sa/checkout-service
  spiffe://prod.example.com/ns/infra/sa/database-proxy
  spiffe://staging.example.com/ns/payments/sa/checkout-service

The trust domain (prod.example.com) scopes trust:
  prod and staging have separate CAs → staging cert rejected in prod
  Cross-trust-domain federation possible (for external partners)
```

SPIRE architecture:

```
SPIRE Server (control plane, 1 per cluster):
  - Root CA or intermediate CA
  - Maintains workload registration entries
  - Issues SVIDs to agents
  - HA: 3-node Raft cluster

SPIRE Agent (data plane, 1 per node):
  - Runs as DaemonSet on every Kubernetes node
  - Attests node identity to Server (via AWS IID, GCP metadata, TPM)
  - Delivers SVIDs to workloads via Unix socket
  - Renews before expiry (1h TTL → renews at 30min)

Workload:
  - Calls Workload API via Unix socket
  - Receives SVID (X.509 cert + private key)
  - Uses for mTLS outbound connections
  - No code changes needed (Envoy/Istio handle transparently)
```

SVID lifecycle:

```
T=0:    Pod starts → Envoy sidecar starts → contacts SPIRE Agent socket
T=0.1s: SPIRE Agent verifies pod identity (Kubernetes service account token)
T=0.2s: Agent requests SVID from SPIRE Server
T=0.5s: SPIRE Server issues X.509 SVID (cert valid for 1 hour)
T=0.5s: Agent delivers cert + private key to Envoy via gRPC stream

T=30min: Agent proactively renews SVID (50% of TTL)
         New cert overlaps old cert (both valid simultaneously)
         Envoy rotates transparently — zero dropped connections

T=1h:   Old cert expires. New cert fully in use.
```

Service Mesh Integration

Istio mTLS modes:

```
PERMISSIVE mode (migration period):
  - Accepts both plain HTTP and mTLS
  - Useful during rollout: old services (plain) + new (mTLS) coexist
  - Danger: plain HTTP accepted means no security guarantee yet

STRICT mode (target state):
  - Rejects any connection without valid client cert
  - All service-to-service calls must present SPIFFE identity
  - Enforce at namespace or mesh level

PeerAuthentication policy:
apiVersion: security.istio.io/v1beta1
kind: PeerAuthentication
metadata:
  name: default
  namespace: payments
spec:
  mtls:
    mode: STRICT   # All inbound connections to payments namespace require mTLS
```

What the Envoy sidecar handles transparently:

```
Without service mesh:
  Application code must: load cert, configure TLS, validate peer cert, handle
rotation

With Istio/Envoy sidecar:
  Application code: makes plain HTTP call to localhost:8080
  Envoy intercepts: wraps in mTLS, presents SVID, validates peer SVID
  Application sees: plain HTTP (TLS is handled at the sidecar layer)

  Rotation: SPIRE pushes new cert to Envoy via xDS API (SDS — Secret Discovery
Service)
```

```
            Envoy hot-reloads cert without dropping connections
            Application is completely unaware
```

Certificate Rotation at Scale

The Rotation Problem

```
100 services × 3 instances each = 300 certs to rotate
Rotation frequency: every 24h (SPIRE default SVID TTL)
= 300 rotations/day = ~0.2 rotations/minute

At Kubernetes scale (1000 nodes, 10K pods):
  10K cert rotations/day = ~7 rotations/minute
  Each rotation: ~5 API calls to SPIRE → 35 SPIRE ops/minute (trivial)
```

Zero-Downtime Rotation Strategy

The overlap window is critical:

```
Wrong approach (causes downtime):
  T=0h:  Issue new cert, invalidate old cert immediately
  Result: In-flight requests mid-handshake fail
          Peer service hasn't yet fetched new cert → rejects connection

Correct approach (overlap window):
  T=-1h: Issue new cert (notBefore = now)
         Old cert still valid (notAfter = T+1h)
  T=0h:  Services rotate to new cert (old still accepted)
  T=+1h: Old cert expires

  During the 1-hour overlap:
    New connections use new cert
    Old connections continue with old cert until they close
    Both certs trusted by all services simultaneously
```

cert-manager (Kubernetes-native rotation):

```
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: checkout-service-cert
spec:
  secretName: checkout-service-tls
  duration: 24h          # cert validity
  renewBefore: 1h        # renew 1h before expiry (overlap window)
```

```
  issuerRef:
    name: spire-issuer
    kind: ClusterIssuer
  dnsNames:
    - checkout-service.payments.svc.cluster.local
  uris:
    - spiffe://prod.example.com/ns/payments/sa/checkout-service
```

What happens during rotation:

```
cert-manager controller watches cert expiry timestamps
At T-1h: requests new cert from SPIRE issuer
         New cert stored in Kubernetes secret (alongside old)

Envoy SDS (Secret Discovery Service):
  Watches Kubernetes secret for changes
  Detects new cert in secret
  Hot-reloads: new cert used for new connections
               old cert used for in-flight connections until close
  No connection drops, no pod restart required
```

Monitoring Certificate Health

Key metrics and alerts:

```
cert_expiry_seconds{service="checkout"} = 3600  # 1 hour left → ALERT

Alert thresholds:
  < 7 days:  Warning (Slack notification, ticket created)
  < 48h:     High severity (escalated, human review)
  < 1h:      Critical (PagerDuty page, auto-remediation triggered)
  < 0:       Cert expired → service TLS handshakes failing → production incident

Prometheus query to find expiring certs:
  cert_expiry_seconds < 86400  # certs expiring in less than 24 hours

Dashboard: count of certs by expiry bucket
  [>30d]: healthy
  [7d-30d]: monitor
  [1d-7d]: warn
  [<1d]: critical
```

Interview Questions

⭐ TLS Fundamentals

A colleague says “we don’t need HTTPS for our internal API — it’s behind a firewall.” How do you respond?

Solution Framework

The firewall fallacy:

```
Firewall protects against external threats.
Internal threats (the more common breach vector) bypass it entirely:
  - Malicious insider
  - Compromised internal host (lateral movement)
  - Developer laptop infected with malware
  - Kubernetes pod escape / container breakout
  - Debug tooling with excessive access
```

What HTTP without TLS exposes internally:

```
1. Confidentiality: Any host on the internal network can read traffic
   → tcpdump / Wireshark on the same subnet = plain text API responses
   → Database passwords, auth tokens, PII all visible

2. Integrity: No protection against modification
   → Compromised internal router can modify responses in flight
   → ARP poisoning attack allows MITM on local subnet

3. Authentication: No server identity
   → Attacker spins up rogue service at same IP (after ARP spoofing)
   → Client connects to attacker's service believing it's the real backend
```

The real cost of “no TLS internally”:

```
Regulatory: PCI-DSS requires encryption in transit for cardholder data,
            regardless of whether traffic is "internal"
            HIPAA: same for PHI (Protected Health Information)

Breach cost: Average cost of a data breach: $4.45M (IBM 2023)
            Cost of deploying internal TLS: Engineering time + cert infra
            ROI: obvious
```

Counter-argument and response:

```
"TLS adds CPU overhead and latency"
→ Modern CPUs with AES-NI: TLS overhead < 1% for typical services
→ TLS 1.3: 1 RTT handshake (only on first connection; reused for subsequent)
→ HTTP/2 multiplexing: many requests over one TLS connection

"Certificate management is complex"
→ cert-manager + Let's Encrypt: automated rotation, zero human touch
→ Istio/Linkerd: mTLS by default with no application code changes
→ Cost: 1 week of engineering, ongoing maintenance near-zero
```

The correct framing:

```
"Defense in depth" — firewall is one layer, not the only layer.
Assume breach: when (not if) an internal host is compromised,
               TLS limits blast radius to that one connection.
               Without TLS: attacker reads all internal traffic from that host.

Recommendation: Use mTLS for all service-to-service. Use TLS 1.3 minimum.
Deploy Istio in STRICT mode so policy is enforced, not optional.
```

 TLS Performance Optimization

Your API handles 50K RPS globally. TLS handshakes are contributing 40ms to your p99 latency. Walk through how you reduce this without compromising security.

Solution Framework

Step 1 — Diagnose where the 40ms goes:

```
TLS 1.2 new session (most expensive):
  TCP handshake:           1 RTT = 20ms (within same region)
  TLS 1.2 handshake:       2 RTT = 40ms
  Total before first byte: 60ms

TLS 1.2 resumed session:
  Session ID / session ticket lookup
  Still requires 2 RTT TLS handshake in TLS 1.2 = 40ms
  (Session resumption in 1.2 only skips cert exchange, not RTTs)

TLS 1.3 new session:
  TCP:             1 RTT = 20ms
  TLS 1.3:         1 RTT = 20ms
  Total:           40ms → saves 1 RTT = 20ms

TLS 1.3 resumed (1-RTT):
  Combined:        1 RTT = 20ms
  Saves 2 RTT vs TLS 1.2 new = 40ms
```

```
TLS 1.3 0-RTT:
  Data in first flight: 0 RTT overhead
  Total: ~5ms (network transmission only)
```

Step 2 — Upgrade to TLS 1.3 (immediate, no security compromise):

```
# NGINX config:
ssl_protocols TLSv1.3;           # Remove TLSv1.2 for new deployments
ssl_session_cache shared:SSL:50m;
ssl_session_timeout 1d;

# TLS 1.3 0-RTT (enable for GET requests only):
ssl_early_data on;
# In upstream proxy_pass block:
proxy_set_header Early-Data $ssl_early_data;
# Backend must check: if Early-Data header present, reject non-idempotent ops
```

Step 3 — Session resumption (TLS 1.3 tickets):

```
TLS 1.3 session tickets:
  Server encrypts session state with ticket key, sends to client
  Client presents ticket on reconnect → 1-RTT handshake

  Session ticket key rotation (security requirement):
    Rotate every 24h → tickets older than 24h require full handshake
    Use ticket key forward secrecy: derive ticket keys from master secret

  Session ticket lifetime:
    Client-side: 7 days maximum (RFC recommendation)
    Server-side: invalidate on server restart or key rotation
```

Step 4 — CDN edge termination (most impactful for global):

```
Without CDN:
  User in Singapore → origin in US: 150ms RTT
  TLS 1.3: 1 RTT = 150ms handshake

With CDN (Cloudflare, Fastly):
  User in Singapore → Singapore PoP: 5ms RTT
  TLS 1.3: 1 RTT = 5ms handshake
  Singapore PoP → origin: persistent connection (already established)

Savings: 150ms → 5ms handshake = 145ms saved per new connection
This is the highest-leverage optimization for global services
```

Step 5 — OCSP Stapling (remove per-handshake CA lookup):

```
ssl_stapling on;
ssl_stapling_verify on;
resolver 8.8.8.8 valid=300s;    # For OCSP fetching
resolver_timeout 5s;

# Result: eliminates ~50-200ms OCSP lookup from client handshake
#         server pre-fetches OCSP response, staples to handshake
```

Step 6 — Certificate size optimization:

```
RSA 4096-bit cert: ~4 KB (larger TLS handshake payload)
ECDSA P-256 cert:  ~1 KB (smaller, faster verification)

ECDSA P-256 advantages:
  128-bit security (same as AES-128)
  Faster signature verification: 0.1ms vs 0.5ms for RSA-2048
  Smaller certificates: faster network transmission

Migration: Issue ECDSA cert alongside RSA cert (dual cert strategy)
           Modern clients (TLS 1.3) get ECDSA; legacy clients get RSA
```

Summary of savings:

```
Baseline (TLS 1.2, global): 40ms TLS overhead
After CDN termination:      5ms  (-35ms, 87% reduction)
After TLS 1.3 upgrade:      3ms  (-2ms,  best with 1-RTT resume)
After OCSP stapling:        3ms  (-0ms,  already counted in CDN)
After ECDSA cert:           2ms  (-1ms,  cert transmission smaller)

Total: 40ms → 2ms = 95% reduction, no security downgrade
```

Key insight: CDN edge termination is the single highest-leverage TLS optimization. It reduces RTT from 150ms cross-continent to 5ms local. All other TLS optimizations combined cannot match this.

 Zero-Trust Service Mesh Security

You are migrating a monolith to 50 microservices. Design a zero-trust security model for service-to-service communication, including identity, authentication, authorization, and certificate lifecycle. The system handles financial transactions.

Solution Framework

Zero-trust principle:

```
Traditional: "Trust everything inside the firewall"
Zero-trust:  "Trust nothing by default. Verify every connection.
```

```
              Assume the network is hostile — even internally."

For financial services:
  PCI-DSS requires: encrypt all cardholder data in transit
  SOC 2 Type II requires: access controls on all service communication
  Zero-trust satisfies both requirements cryptographically
```

Identity layer (SPIFFE + SPIRE):

```
Every service gets a SPIFFE ID tied to its Kubernetes identity:
  spiffe://fintech.prod/ns/payments/sa/checkout-service
  spiffe://fintech.prod/ns/payments/sa/fraud-detection
  spiffe://fintech.prod/ns/ledger/sa/transaction-processor

SPIRE attestation chain:
  Node: attested via AWS IID (Instance Identity Document, signed by AWS)
  Pod: attested via Kubernetes service account projected token
  → Cryptographic proof the workload is what it claims to be

SVID TTL: 1 hour (short = limited blast radius if compromised)
Rotation: automated at 50% TTL (30 minutes), overlapping window
```

Authentication layer (mTLS everywhere):

```
Istio PeerAuthentication — STRICT across all financial namespaces:

apiVersion: security.istio.io/v1beta1
kind: PeerAuthentication
metadata:
  name: strict-mtls
  namespace: payments
spec:
  mtls:
    mode: STRICT

Result:
  Any service without a valid SPIFFE cert rejected at connection
  Rogue pod cannot impersonate a legitimate service
  Compromised pod identity exposed immediately (cert tied to pod SA)
```

Authorization layer (Istio AuthorizationPolicy):

```
Principle of least privilege: each service only allowed to call what it needs

# checkout-service can only call: payment-gateway, fraud-detection, inventory
apiVersion: security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: payment-gateway-access
  namespace: payments
```

```
spec:
  selector:
    matchLabels:
      app: payment-gateway
  rules:
  - from:
    - source:
        principals:
          - "spiffe://fintech.prod/ns/payments/sa/checkout-service"  # ONLY
checkout
    to:
    - operation:
        methods: ["POST"]
        paths: ["/v1/charge", "/v1/refund"]  # ONLY these endpoints

# Any other service calling payment-gateway → denied at sidecar level
# Even if attacker compromises fraud-detection, it cannot call payment-gateway
```

Encryption in depth:

```
Layer 1: mTLS (Envoy sidecar) — encrypts pod-to-pod traffic
Layer 2: Application-level encryption — encrypts sensitive fields
          (card numbers, SSN stored as encrypted blobs in payload)
          Even if mTLS broken: field-level encryption as backstop

Layer 3: Secrets management (Vault or AWS Secrets Manager):
          Database credentials, encryption keys stored in Vault
          Services authenticate to Vault via SPIFFE (Vault JWT auth method)
          Dynamic secrets: Vault generates unique DB credentials per pod
```

Certificate lifecycle for 50 services:

```
Topology:
  SPIRE Server: 3-node Raft HA cluster (critical infrastructure)
  SPIRE Agents: DaemonSet on all nodes (~50 nodes for 50 services)
  cert-manager: Kubernetes controller for non-SPIFFE certs (ingress, etc.)

Rotation math:
  50 services × 3 replicas = 150 SVIDs
  TTL = 1h, rotate at 30min = 2 rotations/hour = ~5/minute
  SPIRE Server load: 5 cert issuances/min (trivial)

Monitoring:
  Alert: SVID not renewed (pod stuck) → cert expiry in <10 minutes
  Alert: SPIRE Server unreachable → agents cannot renew → hard expiry in 30min
  SLO: SPIRE Server availability = 99.99% (more critical than most services)
```

Audit and observability:

```
Every mTLS connection logs:
  Source SPIFFE ID, Destination SPIFFE ID, timestamp, bytes, duration

Enables:
  - Service dependency mapping (who calls whom)
  - Anomaly detection (checkout-service suddenly calling 10 new services?)
  - Compliance reporting (all PCI data flows encrypted and authenticated)
  - Incident response (during breach: trace attacker's lateral movement)

Tooling:
  Istio telemetry → Prometheus → Grafana
  Kiali: real-time service mesh visualization
  Jaeger: distributed tracing (trace ID correlates with mTLS connection logs)
```

Incident response — what zero-trust buys you:

```
Scenario: A pod running checkout-service is compromised

Without zero-trust:
  Attacker moves laterally to payment-gateway, database, everything
  All internal services trusted → full access
  Detection: weeks or months

With zero-trust (SPIFFE mTLS + AuthorizationPolicy):
  Attacker in checkout-service pod can only reach:
    → payment-gateway (authorized)
    → fraud-detection (authorized)
    → NOT database directly (not authorized)
    → NOT other namespaces (not authorized)

  Detection triggers:
    → Unusual API call patterns from checkout-service SPIFFE ID
    → AuthorizationPolicy denials from unexpected source
    → cert-manager: new SVID requested for checkout-service unexpectedly

  Response:
    → Invalidate checkout-service SPIFFE registration in SPIRE
    → All future cert renewals for that service account denied
    → Service can no longer make authenticated calls after current cert expires
(max 1h)
    → Blast radius contained to 1 hour and 2 services
```

Key insights: 1. mTLS alone is authentication, not authorization. You need AuthorizationPolicy to enforce least-privilege access between authenticated services. 2. SPIFFE IDs decouple identity from network address — a service is who its cert says it is, not where it's running. 3. Short SVID TTLs (1 hour) are a security feature: compromise window is bounded even if a private key is stolen. 4. For financial services, layer mTLS + field-level encryption + Vault dynamic secrets — defense in depth assumes any single layer will eventually fail.

Quick Reference: TLS & Encryption Interview Cheat Sheet

When asked about…	Key points to hit
TLS 1.2 vs 1.3	2-RTT vs 1-RTT; 0-RTT resumption; removed weak ciphers; mandatory ECDHE
Forward secrecy	Ephemeral DH key per session; past sessions safe if key leaked; ECDHE
AEAD ciphers	Encryption + integrity in one pass; prevents padding oracle attacks; AES-GCM, ChaCha20
Certificate chain	Root → Intermediate → Leaf; root offline in HSM; intermediate revocable
OCSP stapling	Server pre-fetches OCSP; bundles with handshake; eliminates extra RTT
mTLS	Both sides present certs; cryptographic service identity; no shared secrets
SPIFFE/SPIRE	Workload identity via short-lived X.509; automated rotation; SVID = identity doc
Zero-trust	Trust nothing by default; mTLS + AuthorizationPolicy; assume breach; limit blast radius
Cert rotation	Overlap window = both old and new certs valid; hot-reload in Envoy; monitor expiry
TLS overhead	AES-NI: <1% CPU; latency: 1 RTT (1.3) vs 2 RTT (1.2); CDN termination is biggest win